

第 8 章：事务管理

2026 版

邹兆年

哈尔滨工业大学
计算学部
海量数据计算研究中心
电子邮件: znzou@hit.edu.cn

2026 春

A set of small navigation icons typically found in Beamer presentations, including symbols for back, forward, search, and other slide controls.

教学内容¹

① 8.1 事务的基本概念

- 8.1.1 事务
- 8.1.2 SQL 事务控制语句
- 8.1.3 事务的 ACID 性质

② 8.2 调度的基本概念

3 8.3 异常

④ 8.3 可串行化

- 8.3.1 可串行化调度
- 8.3.2 冲突可串行化调度
- 8.3.3 视图可串行化调度

5 8.4 事务隔离级别

¹课件更新于 2026 年 6 月 1 日

A set of small navigation icons typically found in Beamer presentations, including symbols for back, forward, search, and other slide controls.

8.1 事务的基本概念

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ↺ 🔍 ↻

8.1.1 事务

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ↺ 🔍 ↻

事务 (Transaction)

数据库上执行的一个或多个操作构成的序列，用来完成数据库系统的高级功能

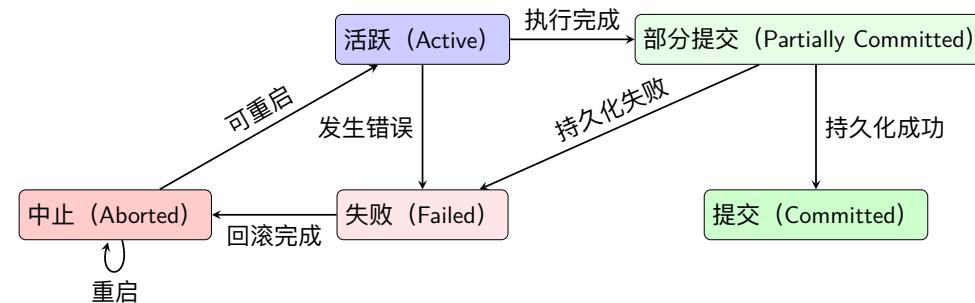
- 事务中的操作**要么全部执行，要么全不执行**

例 (事务)

账户 A 给账户 B 转账 100 元

- ① 查询账户 A 的余额
- ② 如果余额大于等于 100 元，则执行步骤 3 和 4；否则，中止
- ③ 从账户 A 中扣除 100 元
- ④ 在账户 B 中增加 100 元

事务的状态



8.1.2 SQL 事务控制语句

SQL 事务控制语句 (Transaction Control Language, TCL)

事务启动: **BEGIN;** 或 **START TRANSACTION;**

- 启动一个新事务

事务提交 (Commit): **COMMIT;**

- 将事务对数据库的修改持久写入数据库

事务中止 (Abort): **ROLLBACK;**

- 撤销 (Undo) 事务对数据库的全部修改，好像从未执行过该事务
- 事务既可以中止自己，也可以被 DBMS 中止

SQL 事务控制语句 (Transaction Control Language, TCL)

设置保存点 (Save Point): `SAVEPOINT savepoint_name;`

- 设置一个保存点

回滚至保存点: `ROLLBACK TO SAVEPOINT savepoint_name;`

- 回滚至保存点

释放保存点: `RELEASE SAVEPOINT savepoint_name;`

- 撤销保存点

8.1.3 事务的 ACID 性质

事务的 ACID 性质

原子性 (Atomicity): “All or nothing”

- 事务中的操作要么全部执行, 要么全不执行

一致性 (Consistency): “It looks correct to me”

- 如果事务的程序正确, 并且事务启动时数据库处于一致状态 (Consistent State), 则事务结束时数据库一定处于一致状态

隔离性 (Isolation): “As if alone”

- 任意事务的执行不受其他事务的干扰

持久性 (Durability): “Survive failures”

- 事务一旦提交, 则它对数据库的修改一定全部持久写入数据库

原子性 (Atomicity) —“All or Nothing”

任何事务的执行只有两种结局

- 执行完全部操作后提交 $\implies$ 不破坏原子性 ☺
- 执行了部分操作后中止 $\implies$ **破坏原子性** ☹

事务的原子性由 DBMS 保证

- 已中止的事务对数据库的修改必须**撤销 (Undo)**

第 10 章: 故障恢复 (Failure Recovery)

持久性 (Durability) — “Survive Failures”

故障导致任何事务对数据库的修改有 4 种结局

- 已提交事务的修改已全部持久存储 $\Rightarrow$ 不破坏持久性 ☺
- 已提交事务的修改仅部分持久存储 $\Rightarrow$ 破坏持久性 ☹
- 已中止事务的修改有些已持久存储 $\Rightarrow$ 破坏持久性 ☹
- 已中止事务的修改未持久存储 $\Rightarrow$ 不破坏持久性 ☺

事务的持久性由 DBMS 保证

- 重做 (Redo) 已提交事务对数据库的修改
- 撤销 (Undo) 已中止事务对数据库的修改

第 10 章：故障恢复 (Failure Recovery)

◀ ▶ ⏪ ⏩ 🔍 ↺ ↻

邹兆年 (CS@HIT)

第 8 章：事务管理

2026 春

13 / 64

一致性 (Consistency) — “It Looks Correct to Me”

事务的一致性由程序员保证

- 别写错程序

◀ ▶ ⏪ ⏩ 🔍 ↺ ↻

邹兆年 (CS@HIT)

第 8 章：事务管理

2026 春

14 / 64

隔离性 (Isolation) — “As If Alone”

多个事务的执行有两种方式

- 串行执行 (Serial Execution) $\Rightarrow$ 不破坏隔离性 ☺
- 交叉执行 (Interleaving Execution) $\Rightarrow$ 可能破坏隔离性 ☹

事务的隔离性由 DBMS 保证

- 并发控制 (Concurrency Control): 确定多个事务的操作的正确交叉执行顺序

第 9 章：并发控制 (Concurrency Control)

◀ ▶ ⏪ ⏩ 🔍 ↺ ↻

邹兆年 (CS@HIT)

第 8 章：事务管理

2026 春

15 / 64

8.2 调度的基本概念

◀ ▶ ⏪ ⏩ 🔍 ↺ ↻

邹兆年 (CS@HIT)

第 8 章：事务管理

2026 春

16 / 64

基本概念

数据对象 (Data Object) / 数据库元素 (Database Element): 这是个抽象概念, 它可以是属性值、元组、页、关系或数据库

数据库: 固定的数据对象集合 (A fixed set of data objects)

- 我们暂不考虑数据的插入和删除

事务: 数据库对象的读/写操作序列

- 事务可以在数据库上执行很多操作, 但事务管理只关心事务对数据对象的读/写操作

调度 (Schedule)

调度是一个或多个事务的重要操作 (Action) 的序列

例 (调度)

序号	事务 T_1	事务 T_2
1	READ(A, t)	
2		READ(A, s)
3	t := t + 100	
4		s := s * 2
5	WRITE(A, t)	
6		WRITE(A, s)
7	READ(B, t)	
8		READ(B, s)
9	t := t + 100	
10		s := s * 2
11	WRITE(B, t)	
12		WRITE(B, s)

串行调度 (Serial Schedule)

如果一个调度中不同事务的操作没有交叉, 则该调度是串行调度

例 (串行调度)

T_1	T_2	T_1	T_2
READ(A, t)		READ(A, t)	READ(A, s)
t := t + 100		s := s * 2	s := s * 2
WRITE(A, t)		WRITE(A, s)	WRITE(A, s)
READ(B, t)		READ(B, s)	READ(B, s)
t := t + 100		s := s * 2	s := s * 2
WRITE(B, t)		WRITE(B, s)	WRITE(B, s)
	READ(A, s)		
	s := s * 2		
	WRITE(A, s)		
	READ(B, s)		
	s := s * 2		
	WRITE(B, s)		

非串行调度 (Nonserial Schedule)

不是串行调度的调度称作非串行调度

例 (非串行调度)

事务 T_1	事务 T_2	事务 T_1	事务 T_2
READ(A, t)		READ(A, t)	
t := t + 100			READ(A, s)
WRITE(A, t)		t := t + 100	
	READ(A, s)		s := s * 2
	s := s * 2	WRITE(A, t)	
	WRITE(A, s)		WRITE(A, s)
READ(B, t)		READ(B, t)	
t := t + 100			READ(B, s)
WRITE(B, t)		t := t + 100	
	READ(B, s)		s := s * 2
	s := s * 2	WRITE(B, t)	
	WRITE(B, s)		WRITE(B, s)

调度的正确性

事务的一致性保证了单独执行每个事务都不会破坏数据库的一致状态 (Consistent State)

例 (调度的正确性)

设数据库的一致性约束 (Consistency Constraint) 为 $A = B$

事务 T_1	A	B	事务 T_2	A	B
	25	25		25	25
READ(A, t)			READ(A, s)		
t := t + 100			s := s * 2		
WRITE(A, t)	125		WRITE(A, s)	50	
READ(B, t)			READ(B, s)		
t := t + 100			s = s * 2		
WRITE(B, t)		125	WRITE(B, s)		50

串行调度的正确性

任意串行调度都能保持数据库的一致性

例 (串行调度的正确性)

事务 T_1	事务 T_2	A	B
		25	25
READ(A, t)			
t := t + 100			
WRITE(A, t)		125	
READ(B, t)			
t := t + 100			
WRITE(B, t)			125
	READ(A, s)		
	s := s * 2		
	WRITE(A, s)	250	
	READ(B, s)		
	s := s * 2		
	WRITE(B, s)		250

串行调度的正确性

不同的串行调度可能导致数据库处于不同的最终状态, 但都是一致状态

例 (串行调度的正确性)

事务 T_1	事务 T_2	A	B
		25	25
	READ(A, s)		
	s := s * 2		
	WRITE(A, s)	50	
	READ(B, s)		
	s := s * 2		
	WRITE(B, s)		50
READ(A, t)			
t := t + 100			
WRITE(A, t)		150	
READ(B, t)			
t := t + 100			
WRITE(B, t)			150

不正确的非串行调度

非串行调度可能破坏数据库的一致性

例 (不正确的非串行调度)

事务 T_1	事务 T_2	A	B
		25	25
READ(A, t)			
t := t + 100			
WRITE(A, t)		125	
	READ(A, s)		
	s := s * 2		
	WRITE(A, s)	250	
	READ(B, s)		
	s := s * 2		
	WRITE(B, s)		50
READ(B, t)			
t := t + 100			
WRITE(B, t)			150

8.3 异常

异常 (Anomaly)

非串行调度中可能破坏数据库的一致性的动作序列称作异常 (Anomaly)

- 脏写 (Dirty Write)
- 脏读 (Dirty Read)
- 不可重复读 (Unrepeatable Read)
- 幻读 (Phantom)
-

脏写 (Dirty Write)

在事务 T_1 提交前, T_1 写的 A 值被事务 T_2 修改, 后面 T_1 中止

例 (脏写)

T_1	T_2	A
		0
WRITE($A, 1$)		1
	WRITE($A, 2$)	2
ABORT		0

丢失更新 (Lost Update)

在事务 T_1 提交前, T_1 写的 A 值被事务 T_2 修改, 后面 T_1 提交

例 (丢失更新)

T_1	T_2	A
		0
READ(A, s)		
	READ(A, t)	
WRITE($A, s + 1$)		1
COMMIT		
	WRITE($A, t + 1$)	1
	COMMIT	

脏读 (Dirty Read)

在事务 T_1 提交前, T_1 写的 A 值被事务 T_2 读取, 后面 T_1 中止

例 (脏读)

T_1	T_2	A	变量
		0	0
WRITE(A, 1)		1	
	READ(A, t)		$t = 1$
ABORT		0	

不可重复读 (Unrepeatable Read)

- 事务 T_2 修改了事务 T_1 读过的对象 A 的值, 且 T_2 提交
- 如果 T_1 再次读 A 的值, 那么它将得到一个不同于前一次读到的 A 值, 即便 T_1 在此期间并未对 A 进行修改

例 (不可重复读)

设一致性约束条件为 $A \geq 0$

T_1	T_2	A	变量
		1	
READ(A, t)			$t = 1$
	READ(A, s)		
	$s := s - 1$		
	WRITE(A, s)	0	
	COMMIT		
READ(A, t)			$t = 0$
COMMIT			

幻读 (Phantom)

涉及数据的插入或删除, 后面再讲

探究式学习：并发事务的其他异常

通过 LLM 学习

- 并发事务的其他异常
- 异常如何形式化刻画?

8.3 可串行化

8.3.1 可串行化调度

等价调度 (Equivalent Schedule)

如果两个调度在任意数据库实例上都产生相同效果，则这两个调度等价

例 (等价调度)

事务 T_1	事务 T_2	A	B	事务 T_1	事务 T_2	A	B
READ(A, t) $t := t + 100$ WRITE(A, t)		25	25	READ(A, t) $t := t + 100$ WRITE(A, t)		25	25
	READ(A, s) $s := s * 2$ WRITE(A, s)	125			READ(A, s) $s := s * 2$ WRITE(A, s)	125	
			250				125
READ(B, t) $t := t + 100$ WRITE(B, t)			125	READ(B, t) $t := t + 100$ WRITE(B, t)			250
	READ(B, s) $s := s * 2$ WRITE(B, s)		250		READ(B, s) $s := s * 2$ WRITE(B, s)		250

可串行化调度 (Serializable Schedule)

如果一个调度等价于一个串行调度，则该调度是可串行化调度

例 (可串行化调度)

事务 T_1	事务 T_2	A	B	事务 T_1	事务 T_2	A	B
READ(A, t) $t := t + 100$ WRITE(A, t)		25	25	READ(A, t) $t := t + 100$ WRITE(A, t)		25	25
	READ(A, s) $s := s * 2$ WRITE(A, s)	125			READ(A, s) $s := s * 2$ WRITE(A, s)	125	
			250				125
READ(B, t) $t := t + 100$ WRITE(B, t)			125	READ(B, t) $t := t + 100$ WRITE(B, t)			250
	READ(B, s) $s := s * 2$ WRITE(B, s)		250		READ(B, s) $s := s * 2$ WRITE(B, s)		250

可串行化调度的优点

	脏写	脏读	不可重复读	幻读
可串行化	无	无	无	无

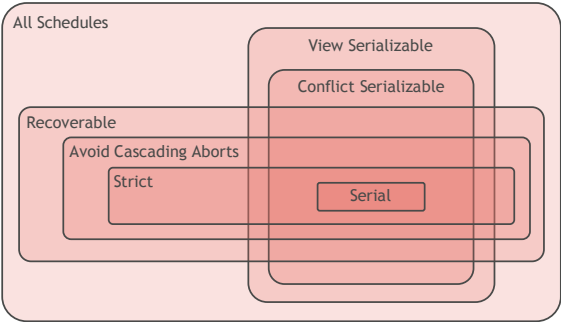
并发事务全部由 DBMS 来调度，程序员无需考虑事务并发的问题

8.3.2 冲突可串行化调度

冲突可串行化（Conflict Serializability）

支持可串行化隔离级别的 DBMS 实施的都是冲突可串行化

- 冲突可串行化比一般的可串行化条件更严格
- 冲突可串行化便于在 DBMS 中实施



冲突（Conflict）

如果两个操作满足以下三个条件，则这两个操作冲突

- 这两个操作属于不同的事务
- 这两个操作涉及相同的对象
- 这两个操作中至少有一个是写

事务的简化表示

对 DBMS 而言, 事务中与调度相关的操作只有对象的读和写

例 (事务的简化表示)

事务 T_1	事务 T_2		事务 T_1	事务 T_2
READ(A, t)			r(A)	
t := t + 100				
WRITE(A, t)			w(A)	
	READ(A, s)			r(A)
	s := s * 2			
	WRITE(A, s)	→		w(A)
READ(B, t)			r(B)	
t := t + 100				
WRITE(B, t)			w(B)	
	READ(B, s)			r(B)
	s := s * 2			
	WRITE(B, s)			w(B)

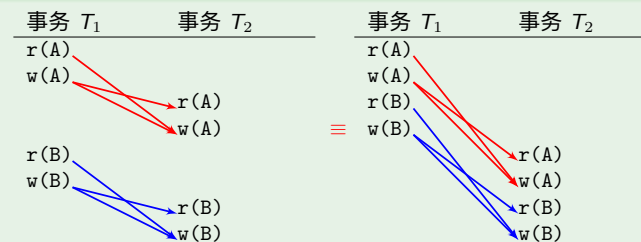
$r_1(A), w_1(A), r_2(A), w_2(A), r_1(B), w_1(B), r_2(B), w_2(B)$. 下标表示事务 ID

冲突等价 (Conflict Equivalence)

两个调度冲突等价 (conflict equivalent), 如果

- 这两个调度涉及相同事务的相同操作
- 每一对冲突的操作在两个调度中的顺序都相同

例 (冲突等价)



冲突可串行化调度 (Conflict Serializable Schedule)

如果一个调度冲突等价于一个串行调度, 则该调度是冲突可串行化调度

例 (冲突可串行化调度)

事务 T_1	事务 T_2		事务 T_1	事务 T_2
r(A)			r(A)	
w(A)			w(A)	
	r(A)			r(A)
	w(A)	≡		w(A)
r(B)			r(B)	
w(B)			w(B)	
	r(B)			r(B)
	w(B)			w(B)

冲突可串行化调度

如果通过将调度 S 中不同事务中的非冲突操作交换顺序可以将 S 转换为一个串行调度, 则 S 是一个冲突可串行化调度

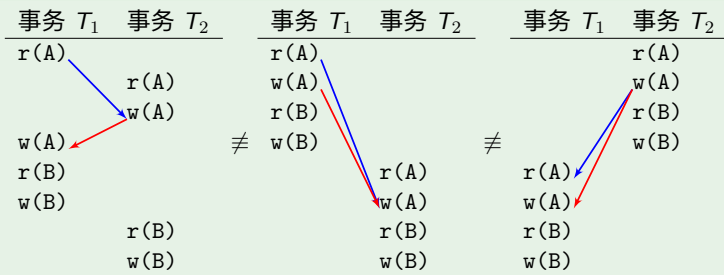
例 (冲突可串行化调度)

T_1	T_2		T_1	T_2		T_1	T_2		T_1	T_2
r(A)			r(A)			r(A)			r(A)	
w(A)			w(A)			w(A)			w(A)	
	r(A)			r(A)			r(A)			r(A)
	w(A)	≡		w(A)			w(A)			w(A)
r(B)			r(B)			r(B)			r(B)	
w(B)			w(B)			w(B)			w(B)	
	r(B)			r(B)			r(B)			r(B)
	w(B)			w(B)			w(B)			w(B)

非冲突可串行化调度

不是冲突可串行化的调度称作非冲突可串行化调度

例 (非冲突可串行化调度)



冲突可串行化测试 (Conflict Serializability Test)

第 1 步：将调度 S 表示为优先图 (Precedence Graph)

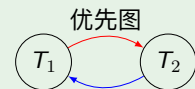
- 每个顶点代表 S 中的一个事务
- 如果事务 T_i 的某个操作 o_i 和事务 T_j 的某个操作 o_j 冲突，并且 o_i 在 S 中先于 o_j ，则优先图中有一条从 T_i 到 T_j 的有向边

第 2 步： S 是冲突可串行化调度，当且仅当其优先图没有环 (Acyclic)

- 优先图顶点的任意拓扑序 (Topological Order) 表示了一个与 S 冲突等价的串行调度

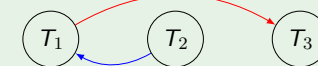
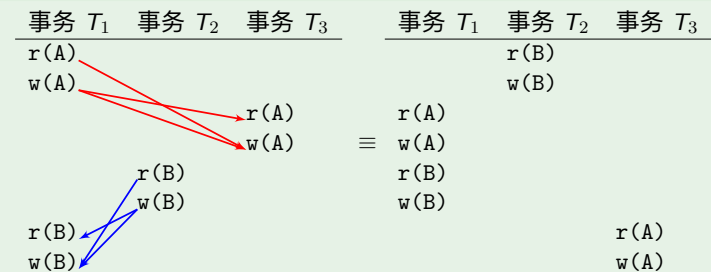
冲突可串行化测试

例 (冲突可串行化测试)



冲突可串行化测试

例 (冲突可串行化测试)

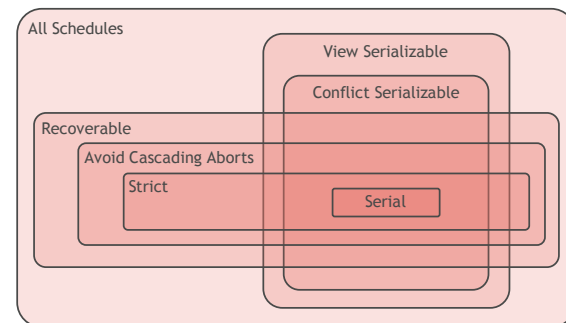


8.3.3 视图可串行化调度

视图可串行化 (View Serializability)

视图可串行化是比冲突可串行化更弱的概念

- 测试和实施视图可串行化很难
- 没有 DBMS 实施视图可串行化



视图可串行化调度 (View Serializable Schedule)

两个调度 S_1 和 S_2 视图等价 (View Equivalent), 如果

- 如果 S_1 中事务 T_1 读了对象 A 的初始值, 则 S_2 中 T_1 也读了 A 的初始值
- 如果 S_1 中事务 T_1 读了事务 T_2 修改过的 A 值, 则 S_2 中 T_1 也读了 T_2 修改过的 A 值
- 如果 S_1 中事务 T_1 写了 A 的最终值, 则 S_2 中 T_1 也写了 A 的最终值

如果一个调度视图等价于一个串行调度, 则该调度是视图可串行化调度

例 (视图可串行化调度)

事务 T_1	事务 T_2	事务 T_3	事务 T_1	事务 T_2	事务 T_3
$r(A)$			$r(A)$		
	$w(A)$			$w(A)$	
$w(A)$					$w(A)$
		$w(A)$			

$\equiv_{view}$

冲突可串行化 vs 视图可串行化

冲突可串行化调度一定是视图可串行化调度

视图可串行化调度不一定是冲突可串行化调度

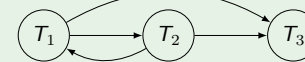
- 视图可串行化但非冲突可串行化的调度一定包含盲写 (Blind Write)

例 (冲突可串行化 vs 视图可串行化)

事务 T_1	事务 T_2	事务 T_3	事务 T_1	事务 T_2	事务 T_3
$r(A)$			$r(A)$		
	$w(A)$			$w(A)$	
$w(A)$					$w(A)$
		$w(A)$			

$\equiv_{view}$

优先图



该调度是视图可串行化调度, 但不是冲突可串行化调度

8.4 事务隔离级别

可串行化调度的优点

	脏写	脏读	不可重复读	幻读
可串行化	无	无	无	无

可串行化调度的缺点

事务隔离级别（Transaction Isolation Level）

可串行化调度的并发度低

SQL-92 规定了并发事务的 4 种隔离级别

在某些应用场景下，并发事务不需要严格隔离

- 读未提交（Read Uncommitted）
- 读提交（Read Committed）
- 可重复读（Repeatable Read）
- 可串行化（Serializable）

- 一个事务对部分对象的修改可以暴露给对其他并发事务
- 弱隔离级别（Weaker Isolation Level）可以提高系统的并发度

在不同隔离级别下，一个事务修改过的对象对其他并发事务的可见程度不同

Read Uncommitted 隔离级别

未提交事务对数据库所做的修改对其他并发事务可见

例 (Read Uncommitted 隔离级别)

事务 T_1	事务 T_2	A	变量
		A = 1	
READ(A, t)			t = 1
	READ(A, s)		s = 1
	s := s * 2		s = 2
	WRITE(A, s)	A = 2	
READ(A, x)			x = 2
	COMMIT		
READ(A, y)			y = 2
COMMIT			
READ(A, z)			z = 2

Read Uncommitted 隔离级别下的异常

隔离级别	脏写	脏读	不可重复读	幻读
READ UNCOMMITTED	无	可能	可能	可能

Read Committed 隔离级别

只有已提交事务对数据库所做的修改对其他并发事务可见

例 (Read Committed 隔离级别)

事务 T_1	事务 T_2	A	变量
		A = 1	
READ(A, t)			t = 1
	READ(A, s)		s = 1
	s := s * 2		s = 2
	WRITE(A, s)	A = 2	
READ(A, x)			x = 1
	COMMIT		
READ(A, y)			y = 2
COMMIT			
READ(A, z)			z = 2

Read Committed 隔离级别下的异常

隔离级别	脏写	脏读	不可重复读	幻读
READ UNCOMMITTED	无	可能	可能	可能
READ COMMITTED	无	无	可能	可能

PostgreSQL 的默认隔离级别为 Read Committed

Repeatable Read 隔离级别

如果一个事务不修改对象 X 的值, 则该事务在提交前任何时候读到的 X 值都等于事务启动时读到的 X 值 [▶ 演示](#)

例 (Repeatable Read)

事务 T_1	事务 T_2	A	变量
		A = 1	
READ(A, t)			t = 1
	READ(A, s)		s = 1
	s := s * 2		s = 2
	WRITE(A, s)	A = 2	
READ(A, x)			x = 1
	COMMIT		
READ(A, y)			y = 1
COMMIT			
READ(A, z)			z = 2

Repeatable Read 隔离级别下的异常

隔离级别	脏写	脏读	不可重复读	幻读
READ UNCOMMITTED	无	可能	可能	可能
READ COMMITTED	无	无	可能	可能
REPEATABLE READ	无	无	无	可能

设置事务隔离级别的 SQL 语句

- 读未提交 (Read Uncommitted): `SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;`
- 读提交 (Read Committed): `SET TRANSACTION ISOLATION LEVEL READ COMMITTED;`
- 可重复读 (Repeatable Read): `SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;`
- 可串行化 (Serializable): `SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;`

总结

- 8.1 事务的基本概念
 - 8.1.1 事务
 - 8.1.2 SQL 事务控制语句
 - 8.1.3 事务的 ACID 性质
- 8.2 调度的基本概念
- 8.3 异常
- 8.3 可串行化
 - 8.3.1 可串行化调度
 - 8.3.2 冲突可串行化调度
 - 8.3.3 视图可串行化调度
- 8.4 事务隔离级别